

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - Python iterációk**
 - Csúszóátlag számítása
 - Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - Vezérlési szerkezetek**
 - Eljárásvezérlés
 - Python fájlkezelés**
 - Fájlkezelés
 - Vezérlési szerkezetek**
 - Eljárásvezérlés
 - Python formátált IO**
 - Python formátált IO
 - Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - Python típusok**
 - Python numerikus típusok
 - Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - Python típusok**
 - További Python típusok**
 - Python function adattípus
 - Absztrakt adattípusok**
 - Összetett adattípusok
 - Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- The diagram illustrates a 26-step learning path for Python, starting from basic introductions and progressing through various programming concepts, data structures, and advanced topics, culminating in a graduation cap icon.

 - 1. Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2. Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3. Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4. Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5. Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6. Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7. Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8. Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9. Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10. Python iterációk**
 - Csőzőátlag számítása
 - 11. Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12. Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13. Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14. Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15. Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16. Python fájlkezelés**
 - Fájlkezelés
 - 17. Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18. Python formátált IO**
 - Python formátált IO
 - 19. Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20. Python típusok**
 - Python numerikus típusok
 - 21. Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22. Python típusok**
 - 23. Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24. Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25. Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26. More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Hanoi tornyai

Problémafelvetés és specifikáció

- Problémafelvetés

- Adjunk megoldást a Hanoi tornyai játékra.
- A játékot Édouard Lucas, egy francia matematikus találta ki. A játék lényege, hogy egy oszlopon egyre csökkenő átmérőjű korongok vannak, és ezeket a korongokat át kell pakolni egy másik üres oszlopra úgy, hogy
 - egyszerre csak egy korongot mozgathatunk valamelyik oszlop tetejéről egy másik oszlop tetejére,
 - nagyobb átmérőjű korong nem kerülhet kisebb átmérőjű korongra, és
 - rendelkezésre áll egy kezdetben üres harmadik oszlop is.

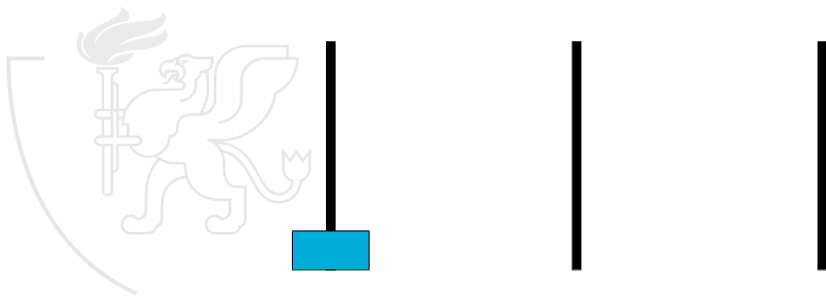
- Specifikáció

- Az input a kezdő és a cél oszlop sorszáma ($\in \{1, 2, 3\}$) valamint a korongok száma (pozitív egész), amelyeket a parancssorból vesszük.
- Egy tevékenységsorozat szövegesen, amit mechanikusan végrehajtva szabályosan átpakolhatjuk a tornyot.

Hanoi tornyai

Algoritmustervezés 1.

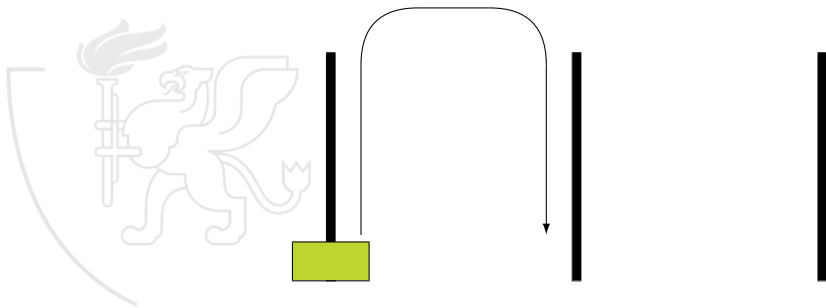
- Készítsünk egy rekurzív eljárást, amelyik az N magasságú torony átpakolását visszavezeti az $N - 1$ magasságú torony átpakolására.
- Az 1 magasságú torony átpakolása nem igényel előkészületet, azonnal elvégezhető.



Hanoi tornyai

Algoritmustervezés 1.

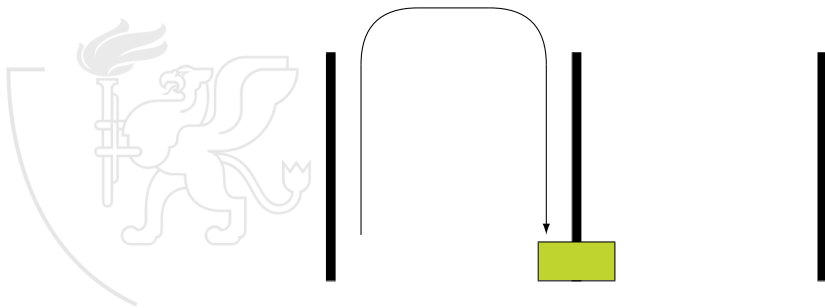
- Készítsünk egy rekurzív eljárást, amelyik az N magasságú torony átpakolását visszavezeti az $N - 1$ magasságú torony átpakolására.
- Az 1 magasságú torony átpakolása nem igényel előkészületet, azonnal elvégezhető.



Hanoi tornyai

Algoritmustervezés 1.

- Készítsünk egy rekurzív eljárást, amelyik az N magasságú torony átpakolását visszavezeti az $N - 1$ magasságú torony átpakolására.
- Az 1 magasságú torony átpakolása nem igényel előkészületet, azonnal elvégezhető.



Hanoi tornyai

Algoritmustervezés 1.

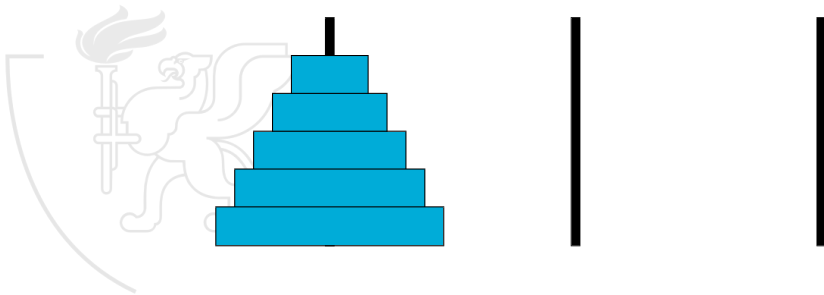
- Készítsünk egy rekurzív eljárást, amelyik az N magasságú torony átpakolását visszavezeti az $N - 1$ magasságú torony átpakolására.
- Az 1 magasságú torony átpakolása nem igényel előkészületet, azonnal elvégezhető.



Hanoi tornyai

Algoritmustervezés 2.

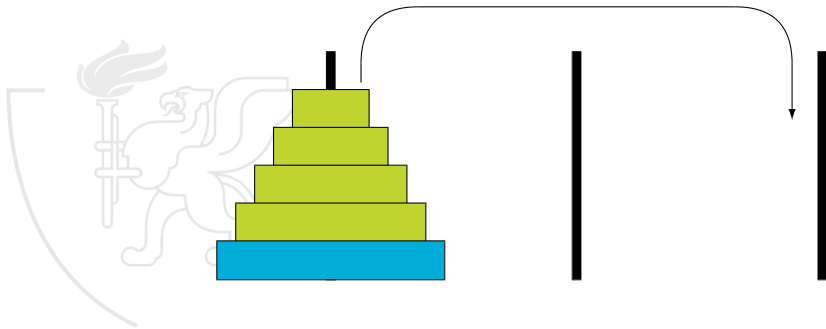
- Az N magasságú torony átpakolását visszavezetjük az $N - 1$ magasságú torony átpakolására.



Hanoi tornyai

Algoritmustervezés 2.

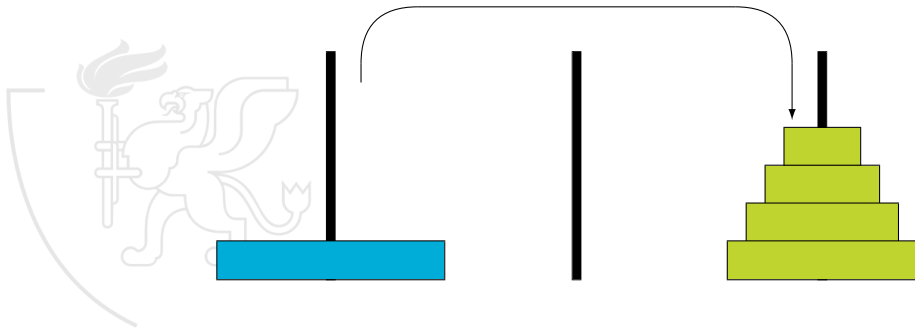
- Az N magasságú torony átpakolását visszavezetjük az $N - 1$ magasságú torony átpakolására.



Hanoi tornyai

Algoritmustervezés 2.

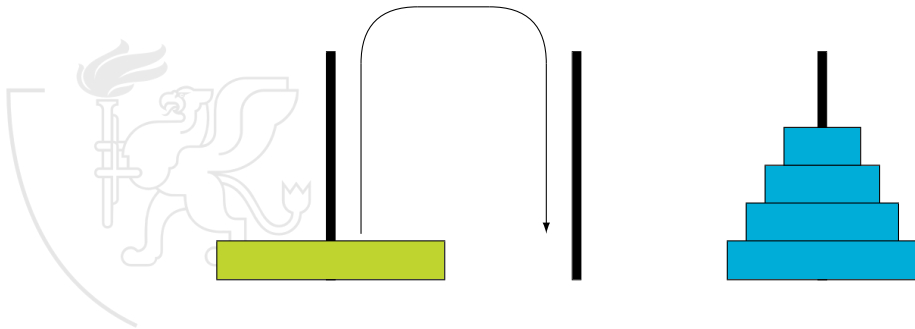
- Az N magasságú torony átpakolását visszavezetjük az $N - 1$ magasságú torony átpakolására.



Hanoi tornyai

Algoritmustervezés 2.

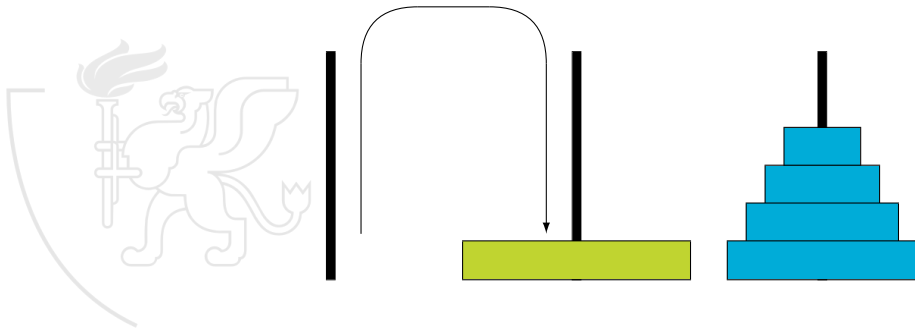
- Az N magasságú torony átpakolását visszavezetjük az $N - 1$ magasságú torony átpakolására.



Hanoi tornyai

Algoritmustervezés 2.

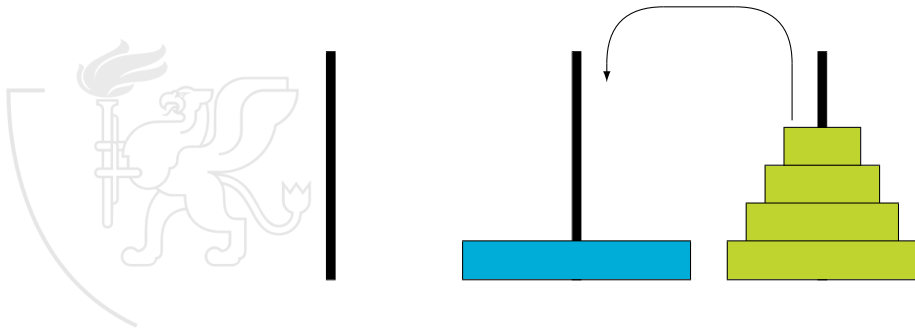
- Az N magasságú torony átpakolását visszavezetjük az $N - 1$ magasságú torony átpakolására.



Hanoi tornyai

Algoritmustervezés 2.

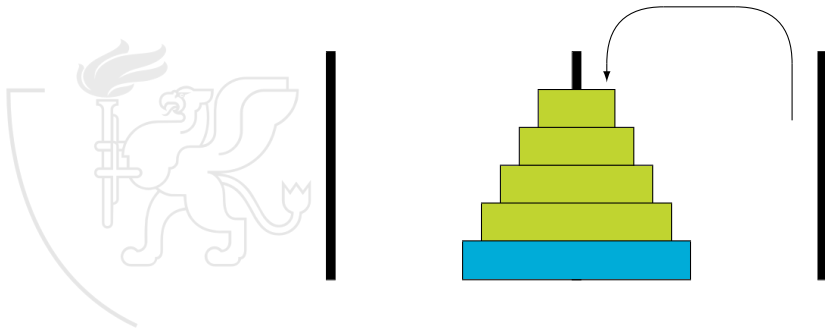
- Az N magasságú torony átpakolását visszavezetjük az $N - 1$ magasságú torony átpakolására.



Hanoi tornyai

Algoritmustervezés 2.

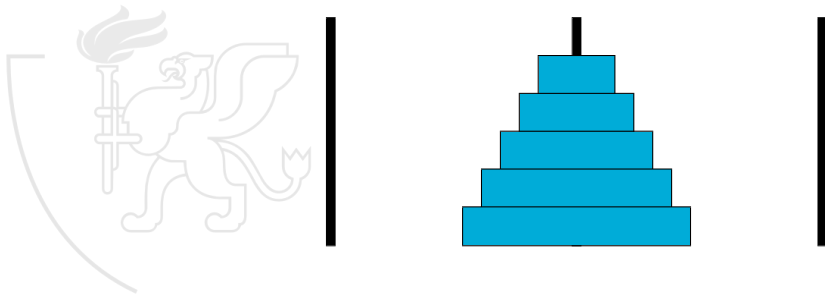
- Az N magasságú torony átpakolását visszavezetjük az $N - 1$ magasságú torony átpakolására.



Hanoi tornyai

Algoritmustervezés 2.

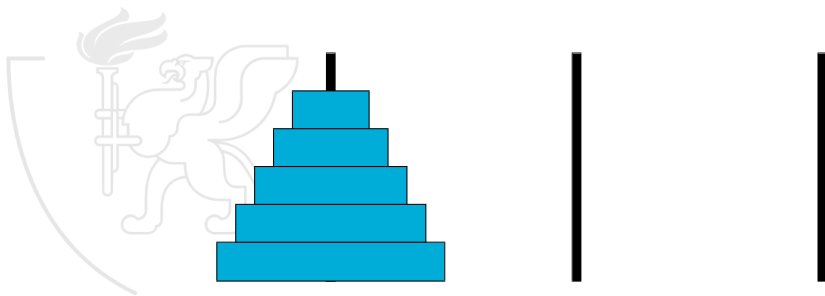
- Az N magasságú torony átpakolását visszavezetjük az $N - 1$ magasságú torony átpakolására.



Hanoi tornyai

Algoritmustervezés 3.

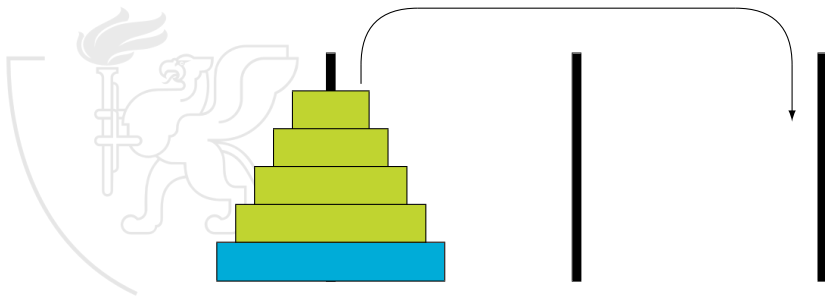
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

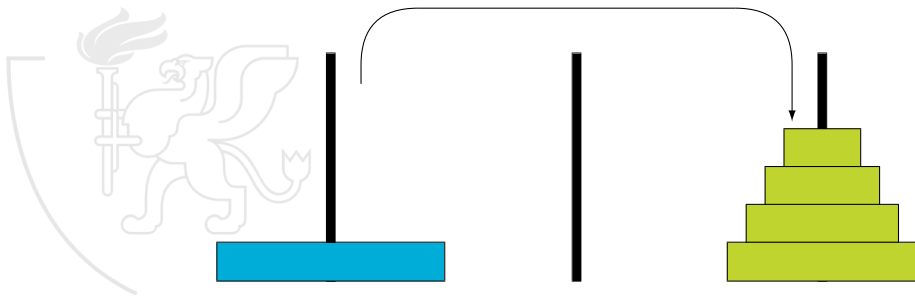
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

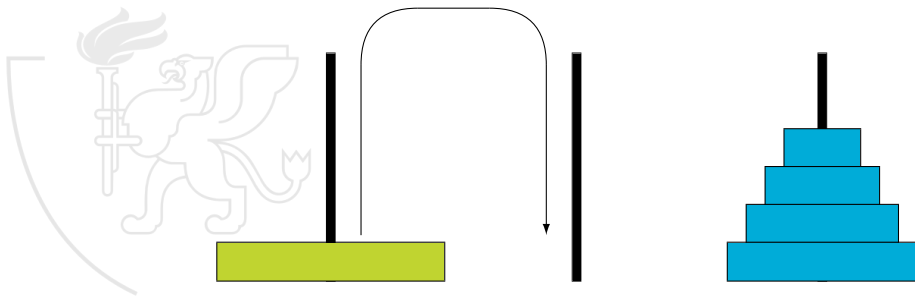
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

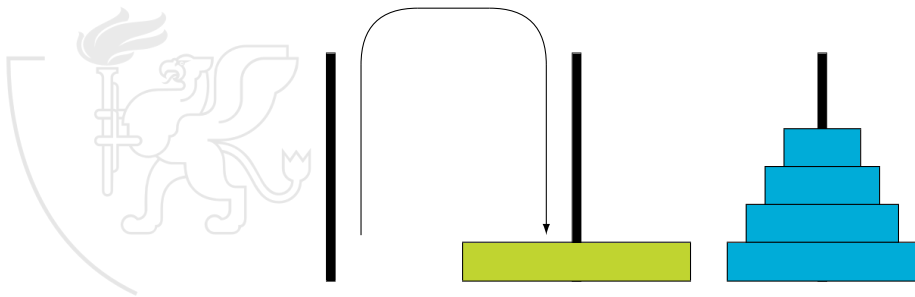
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

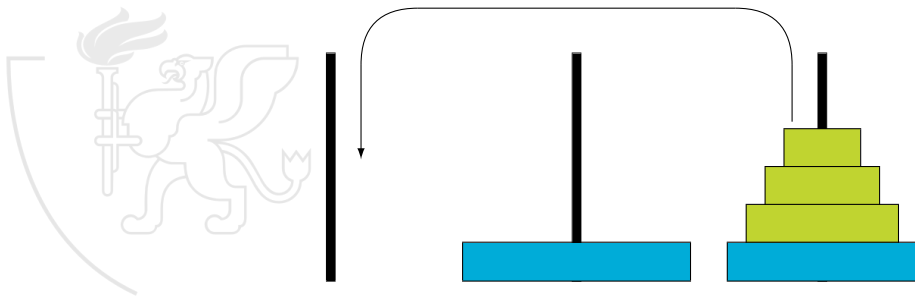
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

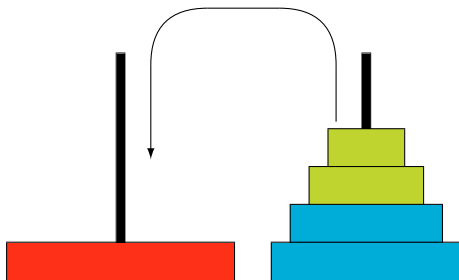
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

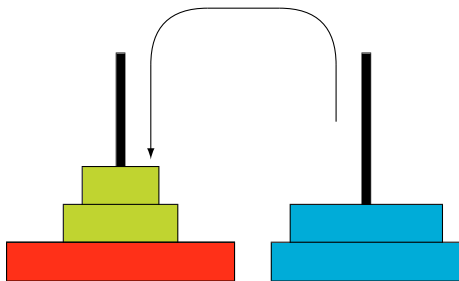
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

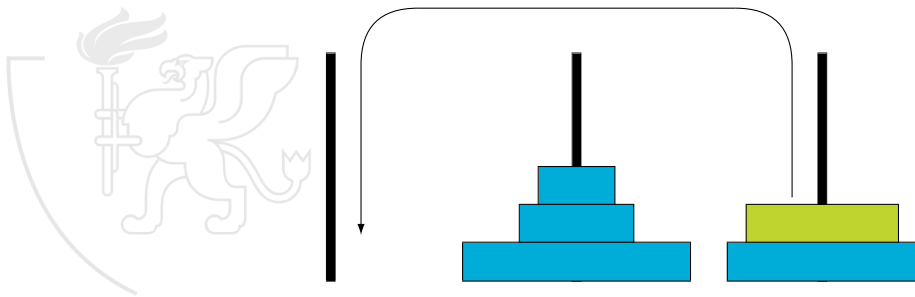
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

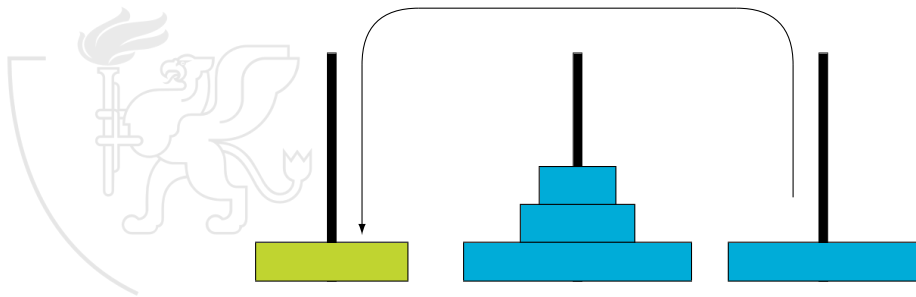
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

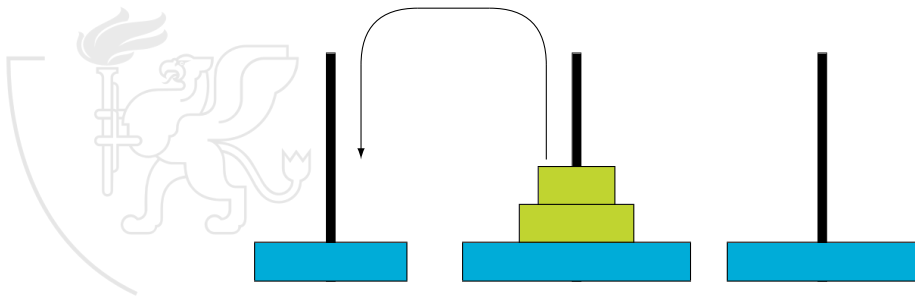
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

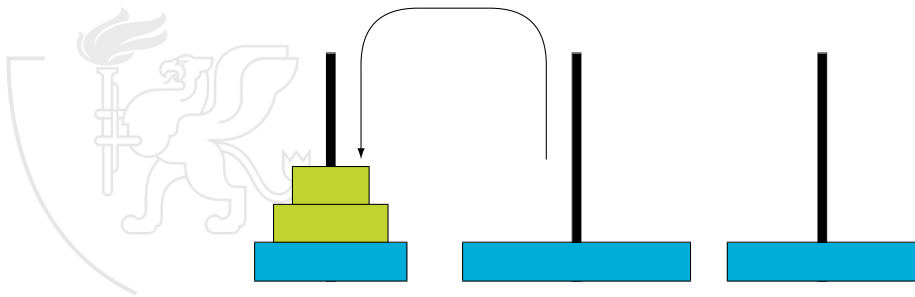
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

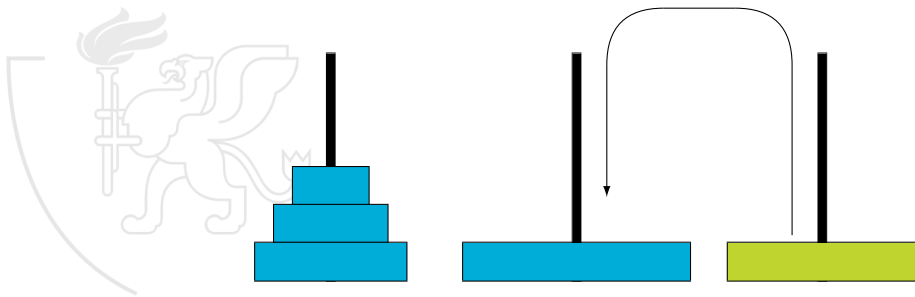
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

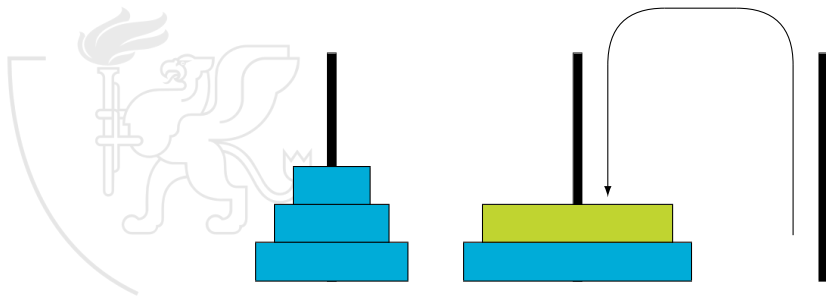
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

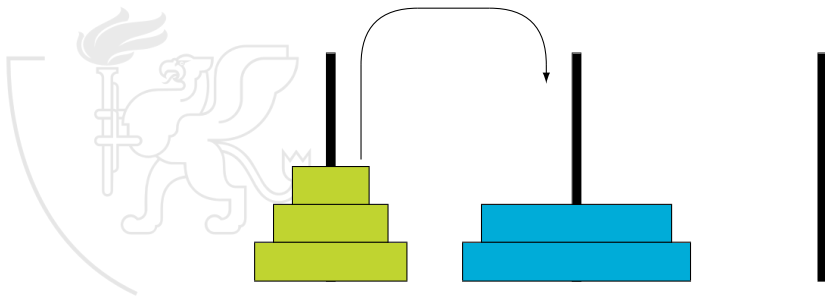
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

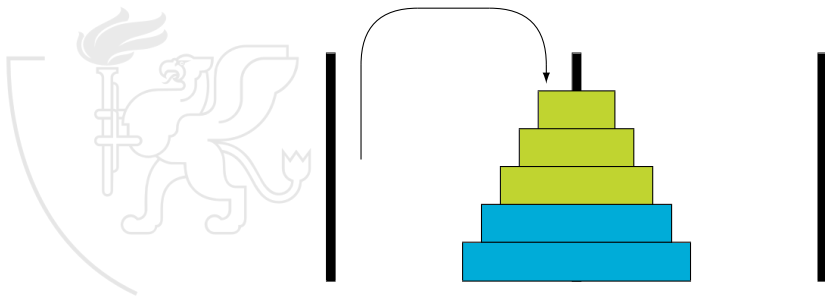
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

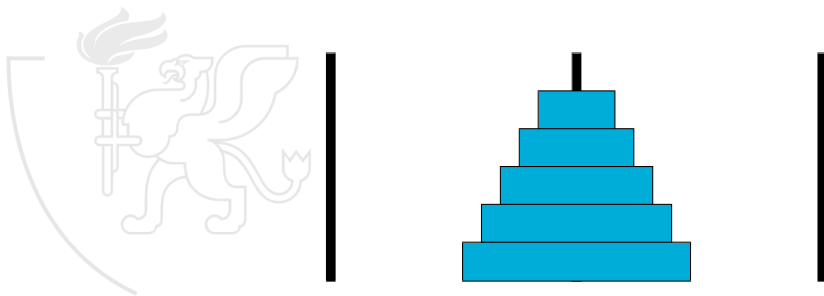
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



Hanoi tornyai

Algoritmustervezés 3.

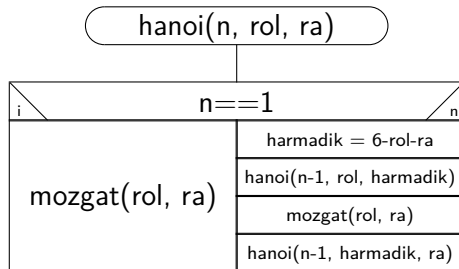
- Mivel az N magasságú torony, amit az egyik rúdról a másikra pakolunk mindig az N legkisebb korongból áll, a harmadik rudat akkor is használhatjuk segédrúdként, ha azon van korong, mivel ez biztosan nagyobb, mint a legnagyobb, amit mi át szeretnénk pakolni.



- A főprogram struktogramja:



- A rekurzív függvény struktogramja:



Parancssori argumentumok kezelése

Az argparse modul [7]

- Az argparse modul segítségével egyszerűbben kezelhetők az összetett paraméterezésű programok argumentumai.
 - Csak a kezelendő argumentumokat és jellemzőiket kell megadnunk, majd a programban könnyedén hozzáférhetünk ezekhez.
 - Kötelező (sorrendi) és opcionális argumentumok kezelése.
 - Automatikus típuskonverzió, lehetséges értékek konfigurálhatósága.
 - Hibás paraméterezés esetén értelmezhető hibaüzenetet ad a programunk (ha mi értelmezhető leírásokat adtunk).



Hanoi tornyai [1/2]

hanoi.py [1-22]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8A Hanoi tornyai feladat megoldása rekurzív algoritmussal.
9"""
10import argparse
11
12def mozgat(innen, ide):
13    print(f"Tegyünk át egy korongot a(z) {innen}. oszlopról a(z) {ide}. oszlopra!")
14
15def hanoi(n, rol, ra):
16    if n == 1:
17        mozgat(rol, ra)
18    else:
19        harmadik = 6 - ra - rol
20        hanoi(n - 1, rol, harmadik)
21        mozgat(rol, ra)
22        hanoi(n - 1, harmadik, ra)
```

Hanoi tornyai [2/2]

hanoi.py [24–40]

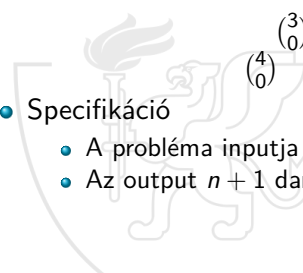
```
24 def main():
25     parser = argparse.ArgumentParser(description = 'A Hanoi tornyai feladat megoldása rekurzív algoritmussal')
26     parser.add_argument('honnan', metavar='HONNAN', default=None, type=int, choices=[1,2,3],
27                          help='Melyik rúdon vannak a korongok?')
28     parser.add_argument('hova', metavar='HOVÁ', default=None, type=int, choices=[1,2,3],
29                          help='Melyik rúdra kell átpakolni a korongokat?')
30     parser.add_argument('mennyit', metavar='MENNYIT', default=None, type=int,
31                          help='Hány korong van?')
32     args = parser.parse_args()
33
34     if args.honnan != args.hova:
35         hanoi(args.mennyit, args.honnan, args.hova)
36     else:
37         print("Kész, hozzá sem kell nyúlni!")
38
39 if __name__ == "__main__":
40     main()
```

Pascal háromszög

Problémafelvetés és specifikáció

- Problémafelvetés

- A Pascal háromszög kiszámolása és megjelenítése.


$$\begin{array}{ccccc} & & \binom{0}{0} & & \\ & \binom{1}{0} & & \binom{1}{1} & \\ & \binom{2}{0} & \binom{2}{1} & \binom{2}{2} & \\ \binom{3}{0} & \binom{3}{1} & \binom{3}{2} & \binom{3}{3} & \\ \binom{4}{0} & \binom{4}{1} & \binom{4}{2} & \binom{4}{3} & \binom{4}{4} \end{array} = \begin{array}{cccccc} & & & 1 & & \\ & & 1 & & 1 & \\ & 1 & & 2 & & 1 \\ 1 & & 3 & & 3 & & 1 \\ 1 & 4 & 6 & 4 & 1 & \end{array}$$

- Specifikáció

- A probléma inputja egy n nemnegatív egész szám.
 - Az output $n + 1$ darab sor piramisszerűen rendezve, rendre a Pascal háromszög sorai.

$\binom{n}{k}$ kiszámítása

Problémafelvetés és specifikáció

- Problémafelvetés
 - Számítsuk ki $\binom{n}{k}$ értékét!
- Specifikáció
 - A probléma bemenete két egész szám, n és k .
 - A probléma kimenete $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ ha értelmezhető n -re és k -ra, különben 0.



$\binom{n}{k}$ kiszámítása

Algoritmustervezés

- Készítsünk az $\binom{n}{k}$ kiszámolására egy rekurzív függvényt.
- Ehhez felhasználjuk, hogy a Pascal háromszög szélén lévő érték 1, a belsejében lévő érték pedig a felette lévő két érték összege.

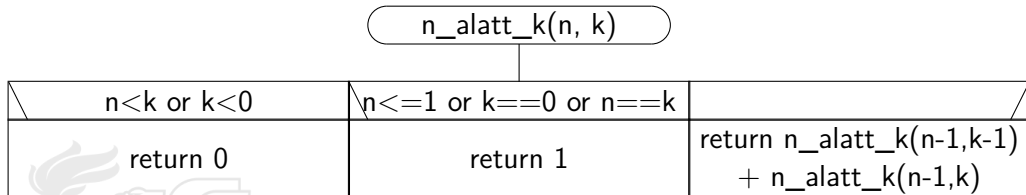
$$\begin{array}{ccc} \binom{n-1}{k-1} & & \binom{n-1}{k} \\ & \searrow \quad \swarrow & \\ & \binom{n}{k} & \end{array}$$



$\binom{n}{k}$ kiszámítása

Algoritmustervezés – Struktogram

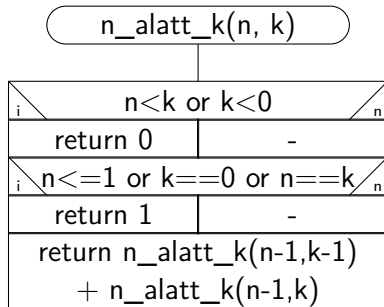
- Az $\binom{n}{k}$ értékét számító rekurzív függvény struktogramja:



$\binom{n}{k}$ kiszámítása

Algoritmustervezés – Struktogram (a return tulajdonságait figyelembe véve)

- Az $\binom{n}{k}$ értékét számító rekurzív függvény struktogramja:



Pascal háromszög rekurzív $\binom{n}{k}$ implementációval

P3szog_r.py [1–14]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Pascal háromszög kirajzolása rekurzív n alatt k függvénnyel
9"""
10from nalattk_r import n_alatt_k
11from P3szog_m import main
12
13if __name__ == "__main__":
14    main(n_alatt_k)
```

Pascal háromszög rekurzív $\binom{n}{k}$ implementációval

P3szog_m.py [1-25]

```
1#!/usr/bin/env python3
2"""
3Programozás_Alapjai_eloadas_peldaprogram
4SZTE_TTIK, Informatikai_Intezet, Szoftverfejlesztés_Tanszék
5Gergely_Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Pascal_háromszög_kirajzolása, main_függvény
9"""
10from os import environ
11
12def main(n_alatt_k):
13    n = int(input("Kérem a Pascal háromszög sorainak számát: "))
14
15    kepsz = int(environ['COLUMNS']) if 'COLUMNS' in environ else 80
16    szamsz = 4 + 2 * (n // 8)
17
18    if 0 <= n <= ((kepsz - 2) // szamsz):
19        for i in range(0, n + 1):
20            sor = "%*d" % (szamsz, n_alatt_k(i, 0))
21            for j in range(1, i + 1):
22                sor += "%*d" % (szamsz, n_alatt_k(i, j))
23            print(sor.center(kepsz - 2))
24    else:
25        print(f"{n} sor nem fér ki a képernyőre!")
```

Pascal háromszög rekurzív $\binom{n}{k}$ implementációval

nalattk_r.py [1–15]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8n_alatt_k kiszámítása rekurzív módon
9"""
10def n_alatt_k(n, k):
11    if n < k or k < 0:
12        return 0
13    if n <= 1 or n == k or k == 0:
14        return 1
15    return n_alatt_k(n - 1, k - 1) + n_alatt_k(n - 1, k)
```

- Az előző példában rekurzív függvénydeklarációt láthattunk, ahol egy függvény saját magát hívta meg.
- A Python nyelven bármelyik függvény lehet rekurzív illetve részt vehet rekurzív függvényrendszerben (ahol a függvények nem feltétlenül közvetlenül hívják meg magukat, hanem esetleg más függvényeken keresztül).



Függvényhívás végrehajtása

- 0.) A program következő feladata egy függvényhívás kifejezés kiértékelése lesz.



a	=	8
b	=	8
c	=	4

```
def f(x, y, z):  
    r = True  
    if z < 0 or x < 1 or y < 1 or  
        8 < x or 8 < y:  
        return False  
    if x == 1 and y == 1:  
        return True  
    z -= 1  
    r = f(x - 2, y - 1, z)  
    r = r or f(x - 2, y + 1, z)  
    r = r or f(x - 1, y - 2, z)  
    r = r or f(x - 1, y + 2, z)  
    r = r or f(x + 1, y - 2, z)  
    r = r or f(x + 1, y + 2, z)  
    r = r or f(x + 2, y - 1, z)  
    r = r or f(x + 2, y + 1, z)  
    return r  
  
a = int(input())  
b = int(input())  
c = int(input())  
c = f(a, b, c)  
print("+" if c else "-")
```

Függvényhívás végrehajtása

- 1.) Memória helyfoglalás a függvény paramétereinek számára és az argumentumok átadása.
 - Az argumentumok Python-ban balról jobbra sorrendben értékelődnek ki. Az argumentumok értéke ezután (referencia szerint) átadódik a függvény formális paramétereinek.



x	=	8
y	=	8
z	=	4
a	=	8
b	=	8
c	=	4

```
def f(x, y, z):  
    r = True  
    if z < 0 or x < 1 or y < 1 or  
        8 < x or 8 < y:  
        return False  
    if x == 1 and y == 1:  
        return True  
    z -= 1  
    r = f(x - 2, y - 1, z)  
    r = r or f(x - 2, y + 1, z)  
    r = r or f(x - 1, y - 2, z)  
    r = r or f(x - 1, y + 2, z)  
    r = r or f(x + 1, y - 2, z)  
    r = r or f(x + 1, y + 2, z)  
    r = r or f(x + 2, y - 1, z)  
    r = r or f(x + 2, y + 1, z)  
    return r  
  
a = int(input())  
b = int(input())  
c = int(input())  
c = f(a, b, c)  
print("+ " if c else "- ")
```

Függvényhívás végrehajtása

- 2.) Függvényhívás
 - A vezérlés átadódik a függvénynek.



x	=	8
y	=	8
z	=	4
a	=	8
b	=	8
c	=	4

```
def f(x, y, z):  
    r = True  
    if z < 0 or x < 1 or y < 1 or  
        8 < x or 8 < y:  
        return False  
    if x == 1 and y == 1:  
        return True  
    z -= 1  
    r = f(x - 2, y - 1, z)  
    r = r or f(x - 2, y + 1, z)  
    r = r or f(x - 1, y - 2, z)  
    r = r or f(x - 1, y + 2, z)  
    r = r or f(x + 1, y - 2, z)  
    r = r or f(x + 1, y + 2, z)  
    r = r or f(x + 2, y - 1, z)  
    r = r or f(x + 2, y + 1, z)  
    return r  
  
a = int(input())  
b = int(input())  
c = int(input())  
c = f(a, b, c)  
print("+" if c else "-")
```

Függvényhívás végrehajtása

- 3.) A függvény törzsének végrehajtása.
 - Elkezdjük végrehajtani a függvény utasításait, egészen az első végrehajtott return utasításig, vagy ennek hiányában a függvény végéig.



x	=	8
y	=	8
z	=	3
r	=	False
a	=	8
b	=	8
c	=	4

```
def f(x, y, z):  
    r = True  
    if z < 0 or x < 1 or y < 1 or  
        8 < x or 8 < y:  
        return False  
    if x == 1 and y == 1:  
        return True  
    z -= 1  
    r = f(x - 2, y - 1, z)  
    r = r or f(x - 2, y + 1, z)  
    r = r or f(x - 1, y - 2, z)  
    r = r or f(x - 1, y + 2, z)  
    r = r or f(x + 1, y - 2, z)  
    r = r or f(x + 1, y + 2, z)  
    r = r or f(x + 2, y - 1, z)  
    r = r or f(x + 2, y + 1, z)  
    return r  
  
a = int(input())  
b = int(input())  
c = int(input())  
c = f(a, b, c)  
print("+" if c else "-")
```

• 4.) Visszatérés

- A függvényblokk formális paraméterei és lokális változói számára foglalt memória felszabadítása.
- A függvényhívás kifejezése felveszi a függvényben kiszámolt értéket.



a	=	8
b	=	8
c	=	False

```
def f(x, y, z):  
    r = True  
    if z < 0 or x < 1 or y < 1 or  
        8 < x or 8 < y:  
        return False  
    if x == 1 and y == 1:  
        return True  
    z -= 1  
    r = f(x - 2, y - 1, z)  
    r = r or f(x - 2, y + 1, z)  
    r = r or f(x - 1, y - 2, z)  
    r = r or f(x - 1, y + 2, z)  
    r = r or f(x + 1, y - 2, z)  
    r = r or f(x + 1, y + 2, z)  
    r = r or f(x + 2, y - 1, z)  
    r = r or f(x + 2, y + 1, z)  
    return r  
  
a = int(input())  
b = int(input())  
c = int(input())  
c = f(a, b, c)  
print("+" if c else "-")
```

Függvényhívás végrehajtása rekurzióval

- 0.) A program következő feladata egy függvényhívás kifejezés kiértékelése lesz.

- A rekurzió is „csak egy” függvényhívás, tehát pontosan ugyanazokat a lépéseket kell végrehajtani.
- Annak, hogy egy már végrehajtás alatt lévő függvényt (rekurzió) vagy másik függvényt hívunk, a függvényhíváskor végrehajtott műveletek tekintetében semmi jelentősége nincs.

x	=	8
y	=	8
z	=	3
r	=	True

a	=	8
b	=	8
c	=	4

```
def f(x, y, z):  
    r = True  
    if z < 0 or x < 1 or y < 1 or  
        8 < x or 8 < y:  
        return False  
    if x == 1 and y == 1:  
        return True  
    z -= 1  
    r = f(x - 2, y - 1, z)  
    r = r or f(x - 2, y + 1, z)  
    r = r or f(x - 1, y - 2, z)  
    r = r or f(x - 1, y + 2, z)  
    r = r or f(x + 1, y - 2, z)  
    r = r or f(x + 1, y + 2, z)  
    r = r or f(x + 2, y - 1, z)  
    r = r or f(x + 2, y + 1, z)  
    return r  
  
a = int(input())  
b = int(input())  
c = int(input())  
c = f(a, b, c)  
print("+" if c else "-")
```

Függvényhívás végrehajtása rekurzióval

- 1.) Memória helyfoglalás a függvény paraméterei számára és az argumentumok átadása.
 - Az argumentumok Python-ban balról jobbra sorrendben értékelődnek ki. Az argumentumok értéke ezután (referencia szerint) átadódik a függvény formális paramétereinek.



x	=	8
y	=	8
z	=	3
r	=	True
x	=	6
y	=	7
z	=	3
a	=	8
b	=	8
c	=	4

```
def f(x, y, z):  
    r = True  
    if z < 0 or x < 1 or y < 1 or  
        8 < x or 8 < y:  
        return False  
    if x == 1 and y == 1:  
        return True  
    z -= 1  
    r = f(x - 2, y - 1, z) ◀◀  
    r = r or f(x - 2, y + 1, z)  
    r = r or f(x - 1, y - 2, z)  
    r = r or f(x - 1, y + 2, z)  
    r = r or f(x + 1, y - 2, z)  
    r = r or f(x + 1, y + 2, z)  
    r = r or f(x + 2, y - 1, z)  
    r = r or f(x + 2, y + 1, z)  
    return r  
  
a = int(input())  
b = int(input())  
c = int(input())  
c = f(a, b, c) ◀  
print("+ " if c else "- ")
```

Függvényhívás végrehajtása rekurzióval

- 2.) Függvényhívás
 - A vezérlés átadódik a függvénynek.



x	=	8
y	=	8
z	=	3
r	=	True
x	=	6
y	=	7
z	=	3
a	=	8
b	=	8
c	=	4

```
def f(x, y, z):  
    r = True  
    if z < 0 or x < 1 or y < 1 or  
        8 < x or 8 < y:  
        return False  
    if x == 1 and y == 1:  
        return True  
    z -= 1  
    r = f(x - 2, y - 1, z)  
    r = r or f(x - 2, y + 1, z)  
    r = r or f(x - 1, y - 2, z)  
    r = r or f(x - 1, y + 2, z)  
    r = r or f(x + 1, y - 2, z)  
    r = r or f(x + 1, y + 2, z)  
    r = r or f(x + 2, y - 1, z)  
    r = r or f(x + 2, y + 1, z)  
    return r  
  
a = int(input())  
b = int(input())  
c = int(input())  
c = f(a, b, c)  
print("+" if c else "-")
```

Függvényhívás végrehajtása rekurzióval

- 3.) A függvény törzsének végrehajtása.
 - Elkezdjük végrehajtani a függvény utasításait, egészen az első végrehajtott return utasításig, vagy ennek hiányában a függvény végéig.



x	=	8
y	=	8
z	=	3
r	=	True
x	=	6
y	=	7
z	=	2
r	=	False
a	=	8
b	=	8
c	=	4

```
def f(x, y, z):  
    r = True  
    if z < 0 or x < 1 or y < 1 or  
        8 < x or 8 < y:  
        return False  
    if x == 1 and y == 1:  
        return True  
    z -= 1  
    r = f(x - 2, y - 1, z)  
    r = r or f(x - 2, y + 1, z)  
    r = r or f(x - 1, y - 2, z)  
    r = r or f(x - 1, y + 2, z)  
    r = r or f(x + 1, y - 2, z)  
    r = r or f(x + 1, y + 2, z)  
    r = r or f(x + 2, y - 1, z)  
    r = r or f(x + 2, y + 1, z)  
    return r  
  
a = int(input())  
b = int(input())  
c = int(input())  
c = f(a, b, c)  
print("+" if c else "-")
```

Függvényhívás végrehajtása rekurzióval

4.) Visszatérés

- A függvényblokk formális paraméterei és lokális változói számára foglalt memória felszabadítása.
- A függvényhívás kifejezése felveszi a függvényben kiszámolt értéket.



x	=	8
y	=	8
z	=	3
r	=	False

a	=	8
b	=	8
c	=	4

```
def f(x, y, z):  
    r = True  
    if z < 0 or x < 1 or y < 1 or  
        8 < x or 8 < y:  
        return False  
    if x == 1 and y == 1:  
        return True  
    z -= 1  
    r = f(x - 2, y - 1, z)  
    r = r or f(x - 2, y + 1, z)  
    r = r or f(x - 1, y - 2, z)  
    r = r or f(x - 1, y + 2, z)  
    r = r or f(x + 1, y - 2, z)  
    r = r or f(x + 1, y + 2, z)  
    r = r or f(x + 2, y - 1, z)  
    r = r or f(x + 2, y + 1, z)  
    return r  
  
a = int(input())  
b = int(input())  
c = int(input())  
c = f(a, b, c)  
print("+ " if c else "- ")
```

- Függvény mellékhatásán azt értjük, hogy a függvényhívás hatására nem csak a függvényérték számíthat ki (és az argumentumok változhatnak meg), hanem megváltozhat egy globális változó értéke is (vagy egyéb műveletek is végrehajtódnak, pl. kiírás).

```
>>> def f(x):  
...     global a  
...     r = x + a  
...     a = x + 1  
...     return r  
...  
>>> f(6) + f(7)  
21  
>>> f(6) + f(7)  
28  
>>> f(7) + f(6)  
29  
>>> f(7) + f(6)  
28  
>>> f(6) + f(7)  
27
```

- Többszörös felhasználás: Hasonló részproblémák megoldására elég egy függvényt készíteni és a különböző adatokra végrehajtani a részalgoritmust. Így a program méretét csökkenteni lehet.
- Memória igény csökkentése: A függvények lokális változói számára csak a függvény végrehajtása idejére foglalódik memória.
- Karbantarthatóság: Függvények használatával a program áttekinthetőbb lesz, ami jelentősen megkönnyíti a későbbi módosítását.
- Modularitás: A tervezés során a részproblémák függvénnyel történő megoldása lehetővé teszi a figyelem lokalizálását.
- Programhelyesség: Függvények alkalmazása megkönnyíti a bizonyítást, a program tesztelését, a hibakeresést és javítást.